

BLOCKCHAIN TECHNOLOGY

5 Most Important Questions - DETAILED ANSWERS

Sinhgad College of Science, Pune | B.Sc. IT (TY) | March 2026

Q1: BLOCKCHAIN ARCHITECTURE & STRUCTURE

Probability: 95% | Marks: 10-12 | Time: 8-10 minutes

QUESTION:

Explain the structure of a blockchain and how blocks are linked together. Include the role of hashing in maintaining the integrity of the blockchain. Draw a diagram showing the relationship between blocks.

DETAILED ANSWER:

A **blockchain** is a distributed, immutable, and decentralized ledger consisting of blocks of data linked together chronologically using cryptographic hashes. Each block contains transaction data and a reference (hash) to the previous block, creating an unbreakable chain where any modification is immediately detectable.

BLOCK STRUCTURE (6 COMPONENTS):

1. **Previous Hash (256-bit):** Cryptographic hash of the previous block's header. This is the key link that chains blocks together sequentially. Even a 1-bit change in the previous block's data produces a completely different hash, breaking the chain.
2. **Timestamp:** Unix epoch time indicating when the block was created/mined. Used to establish chronological order and for network synchronization.
3. **Merkle Root:** A 256-bit hash of all transactions in the block. Created by hashing transaction pairs recursively until a single root hash remains. Allows efficient verification that a transaction is included in the block.
4. **Nonce (Number Used Once):** A 32-bit number miners adjust when performing Proof of Work. Miners try different nonces until finding one that produces a hash meeting the difficulty target.

5. **Difficulty Target:** Sets the mining difficulty. The network adjusts this every 2 weeks (Bitcoin) to maintain consistent block time (~10 minutes for Bitcoin).
6. **Block Hash:** SHA-256 hash of the entire block header. This becomes the 'Previous Hash' in the next block, creating the chain link.

BLOCK BODY (Transaction Data):

Contains all transactions for that block plus their metadata. In Ethereum, also contains smart contract code and state changes. Typically 1,000-2,000 transactions per block in Bitcoin, depending on size constraints (1 MB max).

HOW BLOCKS ARE LINKED TOGETHER:

1. **Linear Chaining:** Each block contains the hash of the previous block. This creates a chronological chain from the genesis block (Block 0) to the most recent block.
2. **Unbreakable Link:** The hash is generated from ALL data in the previous block. Changing even one bit changes the hash completely (avalanche effect). If someone modifies Block 5, its hash changes, breaking the link in Block 6's 'Previous Hash' field.
3. **Cascade Detection:** Modifying Block 5 also requires recalculating Block 6, then Block 7, etc. The attacker must recalculate all subsequent blocks, which becomes computationally infeasible if the blockchain has grown large.
4. **Consensus Verification:** All network nodes maintain a copy of the entire chain. If someone tries to alter past blocks, their version will differ from all other nodes' copies. The majority consensus rejects the fraudulent chain.
5. **Proof of Work Barrier:** For PoW blockchains, modifying past blocks requires redoing the mining work. If it took 10 minutes to mine Block 5 originally, an attacker must redo 10 minutes of work just to recalculate its hash. For 6+ past blocks, this becomes harder than attacking the current block.

ROLE OF HASHING IN BLOCKCHAIN SECURITY:

1. **IMMUTABILITY:** SHA-256 is one-way (irreversible). You can't recover input from output. Any change to data creates a different hash. The blockchain's immutability depends entirely on this property.
2. **TAMPER DETECTION:** The slightest modification to any transaction, timestamp, or data produces a completely different block hash. This cascades to all subsequent blocks, making tampering obvious.

3. **EFFICIENCY:** SHA-256 computation is extremely fast (~4 billion hashes/second on modern hardware). This allows quick verification of the entire chain without excessive computational cost.
4. **SECURITY STRENGTH:** SHA-256 produces 2^{256} possible hashes. Finding a collision (two inputs with same hash) would require $\sim 2^{128}$ attempts, which is computationally infeasible (would take billions of years with current technology).
5. **MINING DIFFICULTY:** PoW relies on hashing. Miners repeatedly hash the block header with different nonces until finding a hash below the difficulty target (e.g., starting with specific number of zeros). This work proves they invested computational resources.

VISUAL DIAGRAM - BLOCKCHAIN STRUCTURE:

BLOCK 0 (GENESIS)

Header:

Previous Hash: 0x000...000 (no prior block)
 Timestamp: 2009-01-03 18:15:05
 Merkle Root: 0x4a5e1e4a...
 Nonce: 2083236893
 Difficulty: 1.0
 Block Hash: 0x6fe28c0a...

Body:

Coinbase Transaction (50 BTC reward)

↓ (Block Hash becomes Previous Hash)

↓

BLOCK 1

Header:

Previous Hash: 0x6fe28c0a... ← LINKED!
 Timestamp: 2009-01-09 02:54:25
 Merkle Root: 0xf3e94742...
 Nonce: 1038288239
 Difficulty: 1.0
 Block Hash: 0x839a8e91...

Body:

Transactions (Tx1, Tx2, Tx3, ...)

↓ (Block Hash becomes Previous Hash)

↓

BLOCK 2

Header:

Previous Hash: 0x839a8e91... ← LINKED!
Timestamp: 2009-01-12 03:30:25
Merkle Root: 0xa7e5ef...
Nonce: 3501879918
Difficulty: 1.0
Block Hash: 0xa16d3637...
Body:
Transactions (Tx4, Tx5, Tx6, ...)

TAMPER DETECTION EXAMPLE:

If attacker tries to change Tx1 in Block 0:

Original Tx1: "Send 10 BTC to Bob"
Modified Tx1: "Send 10 BTC to Eve"

New Merkle Root: Different (because Tx1 changed)
New Block Hash: Completely different

But Block 1 still has Previous Hash: 0x6fe28c0a...
And actual Block 0 hash is now: 0x...NEW

RESULT: Block 1 doesn't match → FRAUD DETECTED
All subsequent blocks break
Network rejects the fraudulent chain

REAL-WORLD EXAMPLE - Bitcoin Block #100:

- Block Height: 100
- Block Hash: 0x6fe28c0ab6f1b372c1a6a246ae63f74f931e8365e15a089c68d6190000000000
- Previous Hash: 0x639a6fd5ed385c16ac14434f7f0f2b23e35b38cfa0d0dcab3e72f1c6000000008
- Merkle Root: 0xc30ce2a5b85f3ecdc2d9fac7e4d7d9e6a8b9c0d1e2f3a4b5c6d7e8f0a1b2c3d
- Timestamp: 2009-01-10 04:33:13 (UTC)
- Difficulty: 1
- Nonce: 2162
- Transactions: 1 (Coinbase only)
- Block Size: 135 bytes

KEY TAKEAWAYS:

- Blockchain = Chronologically linked blocks using cryptographic hashing
- Each block references the previous via its hash → unbreakable chain
- SHA-256 hashing ensures immutability (any change breaks the chain)
- Distributed consensus + cryptographic hashing = Byzantine fault tolerance
- Modifying old blocks requires recalculating all subsequent blocks (exponentially harder)

Q2: CONSENSUS MECHANISMS COMPARISON

Probability: 94% | Marks: 12-15 | Time: 10-12 minutes

QUESTION:

Compare Proof of Work (PoW), Proof of Stake (PoS), and Delegated Proof of Stake (DPoS) consensus mechanisms. What are the advantages and disadvantages of each? Provide real-world examples for each.

DETAILED ANSWER:

1 PROOF OF WORK (PoW) - The Original

DEFINITION: Miners compete to solve complex cryptographic puzzles. The first to solve it gets to add the next block and receives a reward (block reward + transaction fees). Solving the puzzle requires significant computational work, proving the miner invested resources.

HOW IT WORKS: 1. Mempool: Transactions broadcast to network and wait in mempool 2. Block Template: Miners collect transactions, create a block template 3. Mining Puzzle: Find nonce where $\text{SHA256}(\text{block_header} + \text{nonce}) < \text{difficulty_target}$ 4. Brute Force Search: For Bitcoin, miners try ~2.7 trillion hashes per 10 minutes 5. Broadcasting: First miner to find valid hash broadcasts block to network 6. Verification: Nodes verify the hash meets difficulty AND transactions are valid 7. Reward: Miner receives block reward (6.25 BTC in 2024) + transaction fees

ADVANTAGES OF PoW: • **MAXIMUM SECURITY:** To attack Bitcoin, attacker needs 51% of hash power (~\$30 billion in hardware) • **TRUE DECENTRALIZATION:** Anyone with ASIC hardware can participate • **PROVEN TRACK RECORD:** 15 years with no successful 51% attack • **FAIRNESS:** Mining power determines selection, not stake (rich person = equal chance to poor person with efficient ASIC) • **INCENTIVE ALIGNMENT:** Miners profit from network security

DISADVANTAGES OF PoW: • **MASSIVE ENERGY:** Bitcoin ~150 TWh/year (equivalent to Argentina) • **SLOW FINALITY:** 10 minutes/block for Bitcoin, 60+ minutes for payment security • **EXPENSIVE HARDWARE:** ASIC miners cost \$5,000-\$10,000 • **POOL CENTRALIZATION:** Top 3 mining pools control >70% of Bitcoin hash power • **ENVIRONMENTAL:** ~46 megatons CO₂/year for Bitcoin

REAL-WORLD EXAMPLES: • Bitcoin (BTC): 21 million max coins, ~\$1 trillion market cap • Litecoin (LTC): “Silver to Bitcoin’s Gold” • Dogecoin (DOGE): Started as joke, now \$10B+ market cap • Ethereum (before Sept 2022): Used PoW until “The Merge”

2 PROOF OF STAKE (PoS) - The Efficient Alternative

DEFINITION: Validators lock cryptocurrency as “stake” in a smart contract. Network randomly selects validators (weighted by stake amount) to propose blocks. Malicious validators lose their locked coins (slashing penalty).

HOW IT WORKS (ETHEREUM 2.0 EXAMPLE): 1. Staking: User deposits 32 ETH, becomes validator 2. Random Selection: Beacon Chain randomly picks validator (probability = stake) 3. Block Proposal: Validator proposes block within 12 seconds 4. Attestation: Other validators “attest” (vote) block is valid 5. Finality: After 2 epochs (384 seconds), block finalized 6. Rewards: Validator earns ~3-5% APY on staked ETH 7. Slashing: If misbehavior → lose 1-100% of stake

ADVANTAGES OF PoS: • **ENERGY EFFICIENT:** 99.95% less energy than PoW (Ethereum: 0.5 TWh vs 100 TWh) • **FAST FINALITY:** ~13 second finality vs 10 minutes (Bitcoin) or 15 seconds (ETH PoW) • **LOWER BARRIER:** No expensive hardware needed; standard laptop sufficient • **ECONOMIC PENALTY:** Attacking requires 51% stake (~\$50+ billion). Attackers lose their stake • **SIMPLICITY:** No complex hash puzzles; easier to understand and audit

DISADVANTAGES OF PoS: • **WEALTH CENTRALIZATION:** Rich validators earn more rewards proportionally (“rich get richer”) • **HIGH ENTRY COST:** 32 ETH (~\$100,000) to run solo validator • **LESS PROVEN:** Ethereum PoS launched Sept 2022 (still <2 years old) • **SLASHING RISK:** Software bugs can cause validators to lose 1-32 ETH • **CENTRALIZATION IN PRACTICE:** Staking pools (Lido controls 32% of Ethereum stake)

REAL-WORLD EXAMPLES: • Ethereum 2.0 (ETH): Switched Sept 15, 2022. ~600,000 validators, 30+ million ETH staked • Cardano (ADA): Pure PoS from inception (2017), ~3,000 stake pools • Polkadot (DOT): PoS with nominated validators • Solana (SOL): Proof of History (variant of PoS)

3 DELEGATED PROOF OF STAKE (DPoS) - The Democratic Model

DEFINITION: Token holders vote for delegates/validators (typically 21-150). Selected delegates validate blocks. Voters can revoke vote anytime (democratic). Voters earn portion of rewards.

HOW IT WORKS (EOS EXAMPLE): 1. Candidates: Anyone can run for block producer position 2. Voting: Token holders vote for delegates (1 token = 1 vote) 3. Selection: Top 21 become “active” producers, next 71 are “standby” 4. Production: Producers take turns in round-robin fashion 5. Rewards: Shared

proportionally based on votes received 6. Re-voting: Monthly election; bad producers can be voted out

ADVANTAGES OF DPoS: • **HIGHLY SCALABLE:** Only 21-150 delegates vs thousands of validators. EOS: 0.5 sec block time • **ENERGY EFFICIENT:** <0.001% energy of PoW • **DEMOCRATIC:** Continuous elections; bad producers voted out quickly • **FAIR FOR SMALL HOLDERS:** Can participate in voting with small stake • **LOW TECHNICAL REQUIREMENTS:** Just need to vote (no infrastructure)

DISADVANTAGES OF DPoS: • **VOTER APATHY:** Most holders don't vote; ~40-50% participation typical • **CARTEL FORMATION:** Top 21 producers can form agreements to share rewards • **VOTE MANIPULATION:** Whale holders (large stakeholders) can control elections • **POPULARITY CONTESTS:** Marketing matters more than technical capability • **LESS DECENTRALIZED:** Only 21 delegates; easier to attack (11 colluders = 51%) • **POLITICAL DYNAMICS:** Factions form, vote trading, contentious campaigns

REAL-WORLD EXAMPLES: • EOS: 21 elected block producers, 0.5 sec block time • TRON (TRX): 27 super representatives, dominated by exchanges • Cosmos (ATOM): 150 validators • Steem: 20 witnesses (later became HIVE)

COMPREHENSIVE COMPARISON TABLE:

Aspect	PROOF OF WORK	PROOF OF STAKE	DELEGATED PoS
Energy Use	EXTREME: 150 TWh/yr (Bitcoin)	MINIMAL: 0.0055 TWh/yr (ETH)	ULTRA-MINIMAL: <0.001% PoW
Speed	SLOW: 10 min (BTC), 15 sec (ETH PoW)	FAST: 13 sec finality (ETH PoS)	VERY FAST: 0.5 sec (EOS)
Security	HIGHEST: 51% = \$30B+ cost	HIGH: 51% = \$50B+ cost (slashing)	MODERATE: Collusion risk
Decentralization	HIGH: 150,000 nodes	HIGH: 600,000+ validators	LOW: 21-150 delegates only
Entry Barrier	VERY HIGH: \$5-10K ASIC + electricity	HIGH: 32 ETH (~\$100K) or pool	LOW: Just vote
Rewards	NEGATIVE: Cost > reward	3-5% APY	1-3% APY typical
Use Cases	Store of value, settlement	Smart contracts, DeFi, NFTs	Gaming, social networks

Aspect	PROOF OF WORK	PROOF OF STAKE	DELEGATED PoS
Examples	Bitcoin, Litecoin, Dogecoin	Ethereum 2.0, Cardano, Polkadot	EOS, TRON, Cosmos

HISTORICAL EVOLUTION:

• **2009:** Bitcoin launches PoW (Satoshi Nakamoto) • **2012:** Peercoin introduces PoS • **2014:** EOS whitepaper introduces DPoS • **2015:** Ethereum launches using PoW • **2022:** Ethereum switches to PoS (“The Merge”), reducing energy 99.95%

CRITICAL COMPARISON:

Q: Why doesn’t Bitcoin switch to PoS? A: Cultural preference for proven PoW security. Bitcoin prioritizes security over efficiency.

Q: Why did Ethereum switch to PoS? A: Environmental pressure, faster finality, lower barrier to participation. 99.95% energy reduction was primary driver.

Q: Which consensus is “best”? A: Tradeoff. No single best. Security (PoW), Efficiency (PoS), Speed (DPoS). Choose based on blockchain goals.

EXAM ANSWER TEMPLATE:

If asked to compare PoW, PoS, DPoS, structure as: 1. INTRODUCTION (2 lines): Define consensus mechanism 2. PoW SECTION (4 lines): Definition + 2 advantages + 2 disadvantages + 1 example 3. PoS SECTION (4 lines): Definition + 2 advantages + 2 disadvantages + 1 example 4. DPoS SECTION (4 lines): Definition + 2 advantages + 2 disadvantages + 1 example 5. COMPARISON (2-3 lines): Key differences 6. CONCLUSION (1 line): Summarize tradeoffs

Total: ~12-15 lines for complete answer

Q3: CRYPTOGRAPHIC HASH FUNCTIONS (SHA-256)

Probability: 93% | Marks: 10-12 | Time: 8-10 minutes

QUESTION:

What is SHA-256 and why is it critical for blockchain security? Explain the properties of a cryptographic hash function with examples. Why is it impossible to reverse a hash?

ANSWER:

SHA-256 (Secure Hash Algorithm-256) converts any input data into a unique 256-bit (32-byte) hexadecimal string. It's a one-way function - you can hash data but can't reverse it.

PROPERTIES OF CRYPTOGRAPHIC HASH FUNCTIONS:

- 1. DETERMINISTIC:** Same input always produces same output. - Example: SHA256('blockchain') = a4d6f8e2c3b1a9f7d2c5e1a8b3c6d9e2f1a4b7c... - Run it 1 million times: same output every time
- 2. QUICK COMPUTATION:** Fast regardless of input size. - 1 byte → 0.0001 ms - 1 MB → 0.1 ms
- 1 GB → 100 ms
- 3. AVALANCHE EFFECT:** Tiny change produces different output. - 'price is 100 USD' vs 'price is 101 USD' (one character change!) - Hashes completely different (~50% of bits change)
- 4. PRE-IMAGE RESISTANCE (One-Way):** Can't find input from hash.
- Given: Hash = 5feceb66ffc86f38d952786c6d696c79c2dbc238c4cafb11f2271d7a122a7a3e
- Find: Input = ??? - Only option: Try "a", "b", "ab", "abc"... until match -
For 256-bit hash: $\sim 2^{256}$ possibilities (more than atoms in universe)
- 5. COLLISION RESISTANCE:** Nearly impossible to find two inputs with same hash. - Would require $\sim 2^{128}$ attempts - With current tech: billions of years

WHY IS SHA-256 CRITICAL FOR BLOCKCHAIN?

- 1. IMMUTABILITY:** Change 1 bit in transaction → hash changes completely → detected
- 2. FRAUD PREVENTION:** Can't fake transaction without recomputing all subsequent blocks
- 3. PROOF OF WORK:** Mining requires finding hash with difficulty target
- 4. IDENTIFICATION:** Unique identifier for each transaction and block
- 5. EFFICIENCY:** Fast computation allows quick verification

WHY CAN'T SHA-256 BE REVERSED?

Reason 1 - Information Loss: Input can be any size (1 byte to 1 GB), output fixed 256 bits. Reversing would require recovering lost information -

mathematically impossible.

Reason 2 - Brute Force Only: Without mathematical inversion, only option is guess inputs and compare hashes. With 2^{256} possibilities, infeasible.

Reason 3 - Computational Infeasibility: Would take billions of years with current technology.

REAL-WORLD EXAMPLE:

Original Transaction:

From: Alice (public key ABC123)

To: Bob (address XYZ789)

Amount: 1 BTC

Timestamp: 2024-01-15 10:30:45

TX Hash = a7f9c2e1d8b5a3f6c9e2d1a8f7c4b1e8d5a2f9c6e3b0a7d4e1f8c5b2a9f6e

Attacker tries to change: Amount to 10 BTC

Modified Transaction:

Amount: 10 BTC (CHANGED)

New TX Hash = 8c3a1f7e9d4b6a2c5f8e1d3a7c9b6e2f1a8d5c2b9f6e3a0d7c4b1a8f9e6c3

RESULT: Hash completely different! Fraud detected immediately

Q4: TYPES OF BLOCKCHAIN

Probability: 92% | Marks: 10-12 | Time: 10-12 minutes

QUESTION:

Differentiate between Public, Private, and Consortium (Permissioned) blockchains. Provide real-world examples, use cases, and a comparison.

ANSWER:

1. PUBLIC BLOCKCHAIN:

Definition: Open to everyone. No permission needed. All data visible. Decentralized.

Characteristics: - Fully transparent (pseudonymous, not anonymous) - Immutable, decentralized governance - Slower transaction speed - Very high security - Proven track record

Examples: - Bitcoin (BTC): Largest, 21M coin max supply - Ethereum (ETH): Smart contracts, 2M+ validators - Cardano (ADA), Solana (SOL), Litecoin (LTC)

Use Cases: - Cryptocurrency payments (peer-to-peer without banks) - DeFi protocols (lending, trading, yield farming) - NFTs (digital art, collectibles, real estate tokenization) - Public record keeping (certifications, licenses, proof of existence)

2. PRIVATE BLOCKCHAIN:

Definition: Restricted access. Only authorized participants. Centralized control. Data only visible to members.

Characteristics: - Permissioned (need approval) - Centrally controlled - Fast transactions - Energy efficient (often PoA) - Lower operational cost - Less decentralized

Examples: - Hyperledger Fabric: Enterprise blockchain by Linux Foundation - Corda: Financial industry (R3) - JPM Coin: JPMorgan's internal settlements - Quorum: JPMorgan's enterprise version

Use Cases: - Enterprise solutions (banks, insurance, corporations) - Confidential data (medical records, legal documents) - Internal ledgers (cross-department settlements) - Compliance and audit trails

3. CONSORTIUM BLOCKCHAIN:

Definition: Semi-decentralized. Multiple organizations control. Partial permission. Hybrid approach.

Characteristics: - Semi-permissioned (selected organizations) - Multi-party governance (no single entity) - Balanced speed (faster than public, slower than private) - Selective transparency (members see all, public sees limited) - Established trust between participants

Examples: - IBM Food Trust: Retailers (Walmart, Nestlé) tracking food supply - Energy Web Chain: 100+ energy companies tracking carbon credits - Marco Polo: Trade finance network (50+ banks) - SWIFT Integration: Banking consortium

Use Cases: - Supply chain: Manufacturers + distributors + retailers sharing data - Inter-bank settlements: Multiple banks settling transactions - Industry standards: Trade associations verifying data - Sustainability tracking: Companies sharing ESG metrics

COMPARISON TABLE:

Feature	PUBLIC	PRIVATE	CONSORTIUM
Access	Open, no permission	Restricted, invitation only	Selected organizations
Governance	Decentralized (no authority)	Centralized (one authority)	Multi-party (shared control)
Transparency	Fully visible to all	Low (members only)	Partial (members + public aggregate)
Speed	Slow (10 min Bitcoin, 13 sec Ethereum PoS)	Very fast (1-2 sec)	Fast (5-10 sec)
Security	Very high (51% attack = \$30B)	High (trusted nodes, admin control)	High (multiple guardians)
Decentralization	High (thousands of nodes)	Low (centralized)	Moderate (100+ nodes)
Energy	High (especially PoW)	Low (PoA, centralized)	Very low
Entry Barrier	Low (anyone can join)	High (need approval)	High (organizational vetting)
Best For	Cryptocurrency, DeFi	Enterprise, banking	Supply chain, B2B

REAL-WORLD EXAMPLE - Pharmaceutical Supply Chain:

Using PUBLIC Blockchain (e.g., Ethereum): - Advantage: Global transparency anyone can verify drug authenticity - Disadvantage: Competitors see all business details, pricing, margins - Verdict: Poor for competitive industries

Using PRIVATE Blockchain (e.g., Hyperledger): - Advantage: One pharma company controls, keeps trade secrets - Disadvantage: Only works for internal tracking, not trusted by customers - Verdict: Not suitable for accountability

Using CONSORTIUM BLOCKCHAIN (e.g., IBM Food Trust): - Advantage: Pharma companies + regulators + hospitals share data - Hospitals can verify drug authenticity - Regulators audit for compliance - Companies keep manufacturing methods private - Customers see limited info (origin → current location) - Verdict: BEST CHOICE for this use case!

Q5: SMART CONTRACTS & SOLIDITY

Probability: 91% | Marks: 12-15 | Time: 12-15 minutes

QUESTION:

What are smart contracts? Write a Solidity smart contract with data types (uint, string, bool, address), constructor, and at least two functions. Explain the code.

ANSWER:

WHAT ARE SMART CONTRACTS?

Definition: Self-executing programs deployed on blockchain that automatically execute when predetermined conditions are met, without intermediaries.

Key Characteristics:

- **Deterministic:** Same input always produces same output
- **Transparent:** Code and execution visible on blockchain
- **Immutable:** Cannot be modified after deployment (unless designed with upgrades)
- **Autonomous:** Execute automatically without human intervention
- **Cost-effective:** Eliminate intermediaries, reduce fees

REAL-WORLD EXAMPLES:

1. **Insurance Claims:** Auto-trigger payouts when accident documented
2. **Escrow:** Hold payment until both parties fulfill conditions
3. **ICO (Initial Coin Offering):** Auto-distribute tokens to contributors
4. **Supply Chain:** Trigger payment when goods reach destination
5. **NFT Marketplace:** Auto-transfer ownership when ETH received

SOLIDITY CODE EXAMPLE - Simple Bank Account:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleBank {
    // ===== STATE VARIABLES =====
    // Stored permanently on blockchain

    address public owner;           // Owner's Ethereum address
    string public bankName;         // Bank name (text)
    bool public isOpen;             // Is bank operating? (true/false)
    uint public totalDeposits;      // Total amount deposited (256-bit integer)
    uint8 public interestRate;     // Interest rate (8-bit, 0-255)
```

```

// Mapping: account address => balance amount
mapping(address => uint) public accounts;

// ===== CONSTRUCTOR =====
// Runs ONCE when contract deployed

constructor(string memory _bankName, uint8 _interestRate) {
    owner = msg.sender;           // Deployer becomes owner
    bankName = _bankName;         // Set bank name from parameter
    isOpen = true;                // Bank starts open
    interestRate = _interestRate; // Set interest rate
    totalDeposits = 0;            // Start with zero deposits
}

// ===== FUNCTION 1: Deposit Money =====

function deposit() public payable {
    // payable = can receive money (ETH)

    require(isOpen, "Bank is closed!");
    // Check: bank must be open, else reject and show error message

    require(msg.value > 0, "Deposit must be > 0");
    // Check: amount must be positive

    accounts[msg.sender] += msg.value; // Add to user's account balance
    totalDeposits += msg.value;        // Update total (for statistics)
}

// ===== FUNCTION 2: Withdraw Money =====

function withdraw(uint amount) public {
    // amount = how much to withdraw (parameter)

    require(isOpen, "Bank is closed!");
    // Check: bank must be open

    require(accounts[msg.sender] >= amount, "Insufficient balance!");
    // Check: user must have enough balance
    // Example: User has 2 ETH, tries to withdraw 5 ETH → REJECTED

    accounts[msg.sender] -= amount; // Deduct from user's balance
    totalDeposits -= amount;        // Update total

    payable(msg.sender).transfer(amount); // Send ETH to user's wallet
    // payable(...) converts address to payable type (can receive money)
}

```

```

    }

    // ===== FUNCTION 3: Check Balance =====

    function getBalance() public view returns (uint) {
        // view = read-only (doesn't modify blockchain)
        // returns (uint) = returns one unsigned integer
        // view functions are FREE (no gas cost)

        return accounts[msg.sender]; // Return user's current balance
    }

    // ===== FUNCTION 4: Only Owner Can Close =====

    function closeBank() public {
        require(msg.sender == owner, "Only owner can close bank!");
        // Check: Only owner can call this (msg.sender must equal owner)

        isOpen = false; // Set bank status to closed
    }
}

```

CODE EXPLANATION:

DATA TYPES USED:

address owner
 Type: 20-byte Ethereum wallet address
 Visibility: public (readable by anyone)
 Stores: Owner's wallet address (e.g., 0xABC123...)

string bankName
 Type: Text data, variable length
 Visibility: public
 Stores: Name of bank (e.g., "My Bank")

bool isOpen
 Type: Boolean true/false (1 byte)
 Visibility: public
 Stores: Whether bank is operating

uint totalDeposits
 Type: unsigned integer (256-bit by default)
 Range: 0 to $2^{256}-1$
 Visibility: public

Stores: Total money deposited

uint8 interestRate
Type: 8-bit unsigned integer
Range: 0 to 255
Uses less gas than uint (256-bit)
Stores: Interest rate percentage

mapping(address => uint)
Type: Dictionary/hash map structure
Maps: Ethereum address → balance amount
Lookup: O(1) constant time (very fast)
Example: 0xABC123... maps to 5 ETH

CONSTRUCTOR:

Runs ONCE when contract deployed
Initializes all state variables
Parameters: _bankName, _interestRate
Example: deploy("MyBank", 5) sets bankName="MyBank", interestRate=5
memory keyword: only stored during execution, not on blockchain

DEPOSIT FUNCTION:

public payable = anyone can call, can send ETH
require = condition check (if false, reject transaction)
msg.sender = address of function caller (who's depositing)
msg.value = amount of ETH sent
accounts[msg.sender] += msg.value = add ETH to user's balance

WITHDRAW FUNCTION:

parameter "amount" = how much user wants to withdraw
require checks: bank open? user has enough balance?
accounts[msg.sender] -= amount = deduct from balance
payable(msg.sender).transfer(amount) = send ETH back

GETBALANCE FUNCTION:

view = read-only (no gas cost, free to call)
returns (uint) = returns one unsigned integer
Looks up user's balance in accounts mapping and returns it

CLOSEBANK FUNCTION:

require(msg.sender == owner, ...) = only owner can execute
Access control pattern (authorization check)
isOpen = false = bank stops accepting deposits/withdrawals

HOW TO DEPLOY:

1. Go to Remix IDE (remix.ethereum.org)
2. Create new file, paste code
3. Click “Compile” (check mark icon)
4. Click “Deploy” and provide parameters:
 - `_bankName` = “MyBank”
 - `_interestRate` = 5
5. Contract deployed! Now interact:
 - Click “deposit” → send 1 ETH
 - Click “getBalance” → see balance
 - Click “withdraw” → withdraw 0.5 ETH
 - Click “closeBank” → bank closes (only owner)

KEY SOLIDITY CONCEPTS:

Concept	Meaning	Example
Gas	Computational cost to execute	Deposit = ~50K gas
require	Condition check (reverts if false)	<code>require(amount > 0, “error”)</code>
payable	Function can receive cryptocurrency	<code>function deposit()</code> public payable
view	Read-only (no blockchain change)	<code>function getBalance()</code> public view
msg.sender	Address of function caller	<code>owner = msg.sender</code>
msg.value	Amount of ETH sent with call	<code>accounts[msg.sender] += msg.value</code>

EXAM ANSWER TEMPLATE:

If asked about smart contracts, structure as:

1. **DEFINITION** (2 lines): What is a smart contract?
2. **CHARACTERISTICS** (3 lines): Deterministic, Transparent, Immutable, Autonomous
3. **CODE SECTION** (6-8 lines): Provide simple contract code with:
 - State variables (address, string, bool, uint, uint8, mapping)
 - Constructor initialization
 - At least 2 functions with explanations
4. **CODE EXPLANATION** (4-6 lines): Explain what each component does

5. **EXAMPLE/USE CASE** (2 lines): Real-world application
6. **CONCLUSION** (1 line): Summary

Total: ~12-15 lines for complete answer

SUMMARY TABLE - ALL 5 QUESTIONS:

Question	Topic	Key Points	Real Examples
Q1	Blockchain Architecture	Block structure, hashing, linking, tamper detection	Bitcoin Block #100
Q2	Consensus Mechanisms	PoW vs PoS vs DPoS, advantages, disadvantages	Bitcoin, Ethereum 2.0, EOS
Q3	SHA-256 Hashing	Properties, security, irreversibility, immutability	Bitcoin transactions
Q4	Types of Blockchain	Public, Private, Consortium, use cases	Food supply chain
Q5	Smart Contracts	Definition, code, data types, functions, deployment	Bank account contract

EXAM SUCCESS STRATEGY:

Study Priority 1 (40% of exam): Q1 & Q2 - Blockchain fundamentals

Study Priority 2 (35% of exam): Q3 & Q4 - Security & blockchain types

Study Priority 3 (25% of exam): Q5 - Smart contracts & coding

Time Allocation in Exam: - Q1: 8-10 minutes - Q2: 10-12 minutes

- Q3: 8-10 minutes - Q4: 10-12 minutes - Q5: 12-15 minutes

Total: ~50-60 minutes for all 5 questions (out of 90-120 minute exam)

Last Updated: March 17, 2026 **Probability of Appearance:** 90%+ for all 5 questions **Sinhgad College of Science, Pune**